

HabitQuest: Developer Study & Learning Guide

A Technical Breakdown of Zero-Dependency Static PWA & Electron Shell Architectures

Welcome to your comprehensive project study guide! This document is designed as a structured curriculum to help you master how HabitQuest was built from scratch. It explains every architectural layer, design decision, and code module in a clear, educational way.

Part 1: The Tech Stack: Zero-Dependency & Pure Vanilla

Many modern web apps are built with complex frameworks (React, Vue, Angular) and heavy libraries (Chart.js, Tailwind, Webpack). For HabitQuest, we deliberately chose a 100% Zero-Dependency Vanilla Architecture:

- **HTML5 (Structure):** Semantic structure featuring tab panels, native forms, dynamic SVGs, and dialog modals.
- **Vanilla CSS3 (Aesthetic Design):** Flexbox, CSS Grid, animations (@keyframes), viewport media queries, and custom styling variables (--cat-health, --bg-secondary) to build the warm typewriter parchment notebook theme.
- **Vanilla JS / ES6 Modules (System Logic):** Clean, modular scripts divided by concerns using native import and export statements without requiring complex transpilation or bundling.
- **LocalStorage (Database):** Client-side, persistent browser database that operates instantaneously and operates 100% offline.
- **Service Workers (Mobile PWA Shell):** Intercepts network fetch requests, caches static assets, and permits full screen mobile app installations.
- **Electron (Desktop Shell):** A Chromium wrapper that compiles your local web files into a standalone native Windows .exe desktop application.

Part 2: Systems Architecture & File Organization

HabitQuest operates as a Single Page Application (SPA). Instead of reloading pages, it dynamically renders updates directly to the DOM based on a modular structure.

Module Guide

- **index.html** : The static canvas.
- **js/app.js** : App entry point and service worker bootstrapper.
- **js/storage.js** : Data model, backup manager, and state synchronizer.
- **js/gamification.js** : Calculations for leveling, streaks, and award criteria.
- **js/charts.js** : Renders high-fidelity interactive SVG graphs using plain math.
- **js/ui.js** : Listens for UI clicks, updates DOM nodes, and triggers re-renders.

Ø=ÝÃþ Data Management & Storage Engine (storage.js)

All habits and logs are serialized into text strings and stored securely in your browser's physical memory using the LocalStorage API.

```
"id": "hq-9f8d7c6-l3a9z8y",
"name": "Pushups",
"category": "health",
"schemas": { "type": "daily", "days": [] },
"habitId": "hq-9f8d7c6-l3a9z8y",
"createdAt": "2026-05-26",
"date": "2026-05-26",
"completed": true,
"note": "Felt strong! Did 30 reps."
}
### Log Object
export function formatDate(date) {
  const d = new Date(date);
  let month = '' + (d.getMonth() + 1);
  let day = '' + d.getDate();
  const year = d.getFullYear();
  if (month.length < 2) month = '0' + month;
  if (day.length < 2) day = '0' + day;
  return [year, month, day].join('-');
}
```

Ø>Ya Core Concept: Date Normalization

Time zones are a major bug source in calendar trackers. If you log a habit at 11:30 PM, a time zone shift could push the date to tomorrow. HabitQuest resolves this by standardizing all log records to local YYYY-MM-DD strings via a utility function.

Ø=ÜÈ Math & SVG Visualization Engine (charts.js)

Rather than importing a large 200KB graphing framework, HabitQuest draws graphics dynamically using Vector Graphics (SVG).

Today's circular completion Progress:

```
const circumference = 2 * Math.PI * radius; // Approx 277px
// We calculate the circumference of a circle (2 * π * r)
const strokeDashoffset = circumference - (pct / 100) * circumference;
<svg viewBox="0 0 140 140" width="100%" height="100%">
  <circle cx="70" cy="70" r="60" stroke-dasharray="377" stroke-dashoffset="188.5" />
</svg>
```

By binding this inline in an SVG tag:

The browser immediately draws a perfect half-circle completion line without any layout performance drop!

Ø<BÆ Gamification Mechanics (gamification.js)

Gamifying real life keeps motivation high. HabitQuest manages character statistics using algebraic leveling rules:

- **Target XP per level:** Target XP increases by 100 XP per level: $XP_target = Level * 100$
- **Reward Multipliers:** Base completions earn 10 XP. If a user maintains a continuous daily streak, they gain a cumulative bonus to reward consistency: $XP_reward = 10 + \min(25, Streak)$

Ø=Üñ Responsive Layout & Mobile Protection (main.css)

Mobile responsiveness is about flexibility and bounding sizes:

- **Flex Sizing & Bounding (min-width: 0):** Flexbox items default to their minimum content size (min-width: auto). To let long text names wrap cleanly instead of stretching cards off the screen, we enforce min-width: 0; word-break: break-word; white-space: normal; }


```

checkbox-container {
  min-width: 0;
  word-break: break-word;
  white-space: normal;
  max-width: 38px;
  aspect-ratio: 1;
}
```
- **Fluid Aspect-Ratio Checkboxes:** On narrow screens (Samsung S23 portrait), fixed-size checkbox grids overflow. Enforcing an aspect-ratio grid fits perfectly:


```

checkbox-container {
  aspect-ratio: 1;
}
```
- **Overlay Modal Scrolling:** To prevent modal forms from clipping on short viewports, the parent overlay uses overflow: auto; max-height: none; }

& Progressive Web App Caching Logic (service-worker.js)

PWAs install local assets to bypass network requests for offline access. This requires custom caching and update lifecycles:

- **Cache Versioning (CACHE_NAME):** Incrementing the version string forces the browser to discard old assets during the activate event


```

const newWorker = reg.installing;
newWorker.addEventListener('statechange', () => {
  if (newWorker.state === 'installed') {
    window.location.reload();
  }
});
```
- **Ignore Search Matching (ignoreSearch):** Setting { ignoreSearch: true } ignores style version query strings (like ?v=6.0) during runtime matches, keeping files cached for window reloads


```

const newWorker = reg.installing;
newWorker.addEventListener('statechange', () => {
  if (newWorker.state === 'installed') {
    window.location.reload();
  }
});
```
- **Hot-Reload Listener (app.js):** Reloads the browser frame once a new cache is active, pushing design fixes immediately:

